

Pilarius: Multiplexing GPU Memory through Cyclic KV Restoration for Interactive Model Serving

Anonymous Author(s)

Abstract

Emerging interaction models sustain long-lived full-duplex sessions in which input and output overlap. In periodic service, a model processes newly accumulated input in fixed-duration micro-turns and decides whether and when to respond. After a micro-turn finishes, its historical key-value (KV) state can remain idle until the next update, yet still occupy GPU memory. As retained history grows, full residency can exhaust GPU capacity while the combined work of admitted sessions still leaves time within each period. Offloading idle KV frees capacity, but restoring it on demand puts recovery on the next update’s critical path. The periodic schedule provides advance notice of the next update, making it possible to plan restoration before demand. We present the design of PILARIUS, a serving system for memory multiplexing through cyclic KV restoration, without truncating the history retained by the application. Where the application permits, PILARIUS staggers session phases, evicts recoverable KV after use, and schedules its restoration from host memory before the next update. Space released by one session can then accommodate another session’s restoration. The plan limits eviction to the volume that the shared link can restore in time and accounts for each restoration’s full GPU allocation before copying begins.

Placeholder. Evaluation result: PILARIUS serves XX times as many concurrent sessions as full residency and YY times as many as the strongest evaluated baseline [name], under the same service objective and matched generation work. Report the workload, observation duration, and deadline-miss rates.

Keywords: KV cache, GPU memory management, model serving, full-duplex interaction, real-time inference, scheduling

1 Introduction

A full-duplex conversational assistant must continue processing input while speaking so that it can respond to interruptions. MiniCPM-o 4.5, for example, processes input in one-second chunks and decides in each chunk whether to speak [2]. Following Thinking Machines Lab, we call these short interaction intervals *micro-turns* [8]. The model incorporates new input and generates output when needed, reusing history from earlier micro-turns. Serving many such sessions requires managing both their recurring computation and the state that persists between updates.

The memory cost of this state grows with the retained history. Models cache attention keys and values (KV) so that later micro-turns can reuse previously processed history. For MiniCPM-o 4.5, ten minutes of retained input audio alone corresponds to approximately 0.82 GiB of KV per session in BF16. This estimate follows from the model’s 10 audio feature tokens per second and 144 KiB of KV per backbone position [2, 7]. It is a calculation from published parameters, with full retention assumed; it excludes text, visual inputs, control tokens, other model state, and allocation overhead.

Full residency keeps this KV on the GPU even after a micro-turn finishes, when the state is idle until the next update (Figure 1). As histories grow, their combined KV can fill the available memory while all admitted sessions’ work still leaves time within each period. This condition concerns the whole GPU: an individual session’s idle interval may be occupied by another session’s computation. We focus on configurations where KV capacity limits concurrency before the aggregate execution budget is exhausted.

Retaining less history and offloading idle state make different tradeoffs. Metronome bounds the KV window of periodic sessions and uses deadline feedback to control admission [6]. When an application needs a longer history, offloading idle KV to host memory preserves that history but requires recovery before its next use. InferCept, for example, chooses among retaining, swapping, and recomputing state for requests paused by external interactions [1]. In periodic service, eviction between micro-turns makes recovery a recurring obligation. Restoring KV only when the next micro-turn arrives delays that micro-turn’s computation (Figure 2(a)).

Periodic execution provides advance notice of when state will be needed again. The start of each period, or *tick*, is known, so restoration can be scheduled before the next micro-turn. If the application permits phase assignment, the server can also choose each session’s start offset within the period. Staggering these starts creates opportunities for sessions to reuse GPU space as their state becomes idle. We call this reuse *memory multiplexing*: each session retains its own history, but its KV need not occupy GPU space throughout the period.

The difficulty is that restoration itself consumes the space that eviction releases. The server must allocate the entire destination before a copy begins, although the restored KV becomes usable only after the copy finishes. An early restoration therefore occupies memory that other sessions may still need; a later one has less time to finish before the next tick. Evicting more KV also increases the time needed to restore

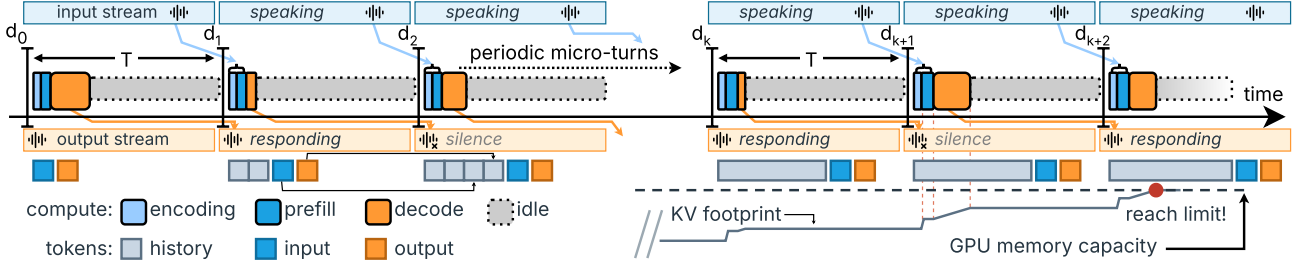


Figure 1. Periodic execution with persistent history. Each micro-turn encodes and prefills new input and may decode output for playback. Retained input and output extend the KV history, which occupies GPU memory even between micro-turns under full residency. The timeline and memory growth are schematic, illustrating how retained state can reach the capacity limit.

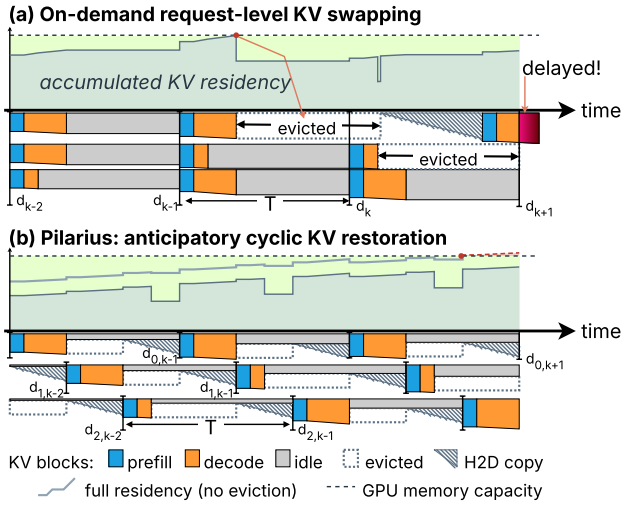


Figure 2. Schematic comparison of restoration timing. (a) On-demand swapping delays computation until KV returns. (b) Staggered sessions release space for one another's restorations before their next ticks. Upper curves show aggregate GPU KV allocation, including each restoration's full destination from the start of allocation. Hatching denotes host-to-device (H2D) copies, dotted outlines denote evicted state, and gray denotes idle resident KV. The pale curve in (b) shows full residency.

it. The question is how to coordinate session phases, eviction amounts, and restoration times so that transfers finish when needed without exceeding GPU capacity.

We present PILARIUS, a design for memory multiplexing through *cyclic KV restoration*. After a micro-turn, PILARIUS evicts part of the session's idle KV and schedules its restoration to finish by the next tick. It coordinates these operations across staggered sessions, arranging for space released by one session to accommodate another's restoration (Figure 2(b)). The design accounts for both the shared transfer schedule and the GPU space available for each restoration. Its goal is to increase concurrency while preserving retained

history and meeting a recurring soft deadline: each micro-turn should finish by the next tick.

Continued history growth requires planning beyond the current period. At admission, PILARIUS considers expected KV growth, computation, transfers, and GPU allocation over a finite lookahead interval. The resulting plan is reused across micro-turns and revised infrequently as histories grow or sessions enter and leave. This planning anticipates changes in resource demand; feasibility within the lookahead interval does not guarantee capacity for an entire session.

Placeholder. Final contributions and measured results: identify the configurations where KV capacity limits concurrency while compute time remains; establish the benefit attributable to coordinating phases, eviction amounts, and restoration timing; report concurrency and deadline-miss rates against full residency and on-demand restoration under matched service objectives.

2 Background and Motivation

2.1 Periodic Execution and Retained State

An application's retention policy determines which history remains available to the model. Full retention, a fixed window, and a summary therefore have different implications for both memory use and model behavior. Residency management preserves the selected history while changing where its KV is stored. Reclaiming GPU space requires an interval without access to that state; periodic input capture alone does not establish such an interval.

For timing analysis, consider sessions with a common period T and reference time t_0 . Session i has phase $\phi_i \in [0, T)$, so its micro-turn k is released at the tick

$$r(i, k) = t_0 + \phi_i + kT. \quad (1)$$

Let $c(i, k)$ denote completion of generation for that micro-turn. Its soft deadline is the next tick, giving the service target

$$c(i, k) \leq r(i, k + 1) = r(i, k) + T. \quad (2)$$

Occasional misses are permitted under a specified deadline-miss rate limit. Latency is $c(i, k) - r(i, k)$, so delays before

submission remain part of the service latency and do not shift the deadline. Initial phase alignment and delivery delay require separate accounting (Section 5.1).

Placeholder. Allowed deadline-miss rate, observation interval, and model-specific generation-completion event.

Reclamation can begin only after computation stops accessing the selected KV. Let $q(i, k)$ denote the last use of the KV selected for eviction in micro-turn k . The server can release that state once outstanding references are gone and a valid host copy is available. Restoration must complete before the next computation accesses the state; PILARIUS targets the next tick, $r(i, k + 1)$, to avoid delaying that computation. The planned reclamation and restoration window therefore extends from $q(i, k)$ to $r(i, k + 1)$, provided no intervening accesses occur. Late computation shortens this window and may eliminate it entirely.

2.2 When KV Capacity Limits Concurrency

The capacity argument compares the memory and execution time required by all admitted sessions. One session's KV may be idle while other sessions keep the GPU busy, and periodic releases alone do not establish compute slack [5]. Execution time can also increase with context length, even when the amount of new input and generated output per micro-turn is fixed. We must therefore assess the complete workload's execution time at the context length that fills GPU KV capacity.

Consider N sessions whose inputs are available at the same tick, with a common period T and no inter-session KV sharing. Each session reaches a maximum retained length of L positions within the period, with κ KV bytes per position. Let G_{KV} be the capacity available for KV after weights, activations, and other runtime allocations. Ignoring allocation rounding, full residency fills this pool at

$$L_{\text{mem}} = \frac{G_{KV}}{N\kappa}. \quad (3)$$

Let $C_N(L)$ be the elapsed time to execute all N sessions' per-period work under a specified policy, with the required KV resident. This includes input processing, model execution, and scheduling overhead; batching means it is not generally the sum of isolated session times.

The remaining compute budget at the memory limit is $T - C_N(L_{\text{mem}})$. A positive value means that the retained KV fills the pool before the complete workload uses up the period. To obtain a sufficient condition in terms of resource and workload parameters, suppose execution time satisfies

$$C_N(L) \leq A_N + B_N L, \quad A_N \geq 0, B_N \geq 0, \quad (4)$$

for fixed per-period input and generation work over the context range under analysis. The intercept A_N accounts for work independent of history length, while the slope B_N allows cost to increase with that length. This affine envelope

must be established as an upper bound for the chosen workload, platform, and execution policy. If L_{mem} lies within the same range, then

$$T - C_N(L_{\text{mem}}) \geq T - A_N - \frac{B_N G_{KV}}{N\kappa}. \quad (5)$$

A positive right-hand side is sufficient to establish compute slack at the memory limit. If it is at least ηT , for $0 < \eta < 1$, the workload leaves at least a fraction η of the period unused.

Fitting A_N and B_N to measurements does not establish an upper bound. Such a fit predicts slack and requires validation on independent configurations. Even a valid slack bound does not guarantee that additional sessions or recomputation meet their deadlines. Their work must fit alongside recovery costs and interference while preserving every session's deadline.

Placeholder. Motivating measurement: joint KV allocation and aggregate execution time at the full-residency limit, with concurrency and context length; validate the predicted compute slack on separate configurations.

2.3 Restoration Needs Both Time and Space

Eviction frees GPU capacity until the server allocates a destination for restoration. The entire destination must be allocated before copying begins, even though its contents are not yet usable. Let $\mathcal{P}(t)$ contain the physical GPU blocks allocated to KV at time t , including destinations of unfinished restorations. For block sizes b_p , the capacity constraint is

$$\sum_{p \in \mathcal{P}(t)} b_p \leq G_{KV} \quad \text{at every } t. \quad (6)$$

Each shared physical block is counted once; counting only valid KV would omit space already committed to ongoing restorations.

The destination must become available early enough to complete the transfer before the next tick. For a restoration of e bytes starting at u , let $\widehat{B}$ be the estimated effective host-to-device bandwidth. With an allowance δ for scheduling and completion overhead, the estimated completion must satisfy

$$u + e/\widehat{B} + \delta \leq r(i, k + 1). \quad (7)$$

The bandwidth estimate must reflect the shared transfer schedule, including interference from host backup traffic and concurrent GPU work. The nominal link rate alone does not capture these effects.

Equations 6 and 7 constrain the same restoration decision. Starting earlier leaves more time for copying but holds the destination longer; evicting more bytes releases more capacity but lengthens the transfer. Session phases determine when memory is released relative to when other sessions need their KV restored. A per-period transfer budget can therefore be satisfied even when individual restorations lack time or their destinations cannot coexist in GPU memory.

The server must check the transfer windows and GPU allocation over time, together with computation under the chosen phases.

3 PILARIUS Design

3.1 Overview

PILARIUS separates planning from the repeated execution of a residency schedule. The session manager plans at admission, follows the plan across micro-turns, and revises it as accumulated growth or changes in the session population require. Its decisions are each session's phase, eviction budget, and restoration timing. The plan seeks to free GPU KV capacity while leaving enough time for transfers and computation before the recurring deadlines.

Consider the one-second session in Figure 1. At a tick, its previous history is ready on the GPU. The session processes the new input and generates any output, extending that history. After the selected KV ranges are no longer in use and have valid host copies, the manager can release their GPU storage. Other sessions use the reclaimed capacity while this session is idle. Before its next tick, the manager allocates the restoration destination and copies the evicted ranges back. Figure 2(b) shows this cycle across staggered sessions.

3.2 Admission Planning

Phase assignment. For a period T , the manager divides the period into S uniformly spaced slots with phases

$$\phi_j = jT/S, \quad j \in \{0, \dots, S-1\}. \quad (8)$$

An arriving session receives one of these phases. Uniform spacing spreads computation and restoration across the period. It does not by itself guarantee balanced traffic: sessions may have different histories, computation times, and eviction budgets.

Planning horizon. Admission uses the configured model and period, current retained state, expected state growth, and resource estimates over a finite horizon H . The aim is to anticipate the gradual increase in memory and recovery demand rather than wait for an allocation failure. A plan feasible over H does not reserve capacity for every admitted session to reach its maximum context length. Continued service therefore depends on extending or revising the plan as execution proceeds.

Per-session budgets. The eviction budget specifies how many bytes of a session's idle KV may be released. The manager must choose it together with restoration timing: a larger budget is useful only if those bytes can return before the next tick. Budgets may differ across sessions, but their feasibility depends on the shared resources. Admission must account for computation under the chosen phases, transfers within their available windows, host storage, and GPU allocation at every point in the horizon. In particular, it must count

the full destination of each restoration when that space is allocated (Equation 6).

A free phase slot is consequently insufficient for admission. A candidate schedule may fit its total traffic within one period and still fail because several destinations must coexist before another session can release memory. Conversely, making every eviction small enough to avoid transfer contention may leave too much idle state on the GPU. The planning problem is to find a useful eviction budget for each session while satisfying both constraints.

Placeholder. Admission algorithm: selection of S and H , growth and resource estimates, phase-slot assignment, budget allocation, scheduling margins, and the feasibility test.

3.3 Cyclic KV Restoration

Releasing idle state. Eviction begins only after the selected ranges are no longer referenced by computation or ongoing transfers. For host-backed restoration, each range must also have a complete, valid host copy. An unfinished backup is not a recovery source, and making a backup does not itself release the GPU allocation. If a physical block is shared, all outstanding references must permit reclamation.

The budget limits the amount reclaimed; it does not require eviction of the session's entire history. State that cannot be recovered within the planned window remains resident. This distinction matters as histories grow: even with a fixed restoration budget, the resident portion may increase and eventually require a plan revision.

Placeholder. KV range selection and block granularity; creation and validation of host copies, including the cost of backing up newly generated state.

Starting restoration. The manager schedules H2D copying to finish before the next tick, allowing for transfer and completion overhead (Equation 7). A restoration starts only when its planned start time has arrived and enough GPU memory is available for its full destination. Space released by another session's eviction can satisfy this allocation. Memory availability alone does not trigger an early copy: holding restored state longer than necessary can reduce the capacity available to other sessions.

The H2D stream is shared, so a session cannot independently assume the entire link rate throughout its restoration window. Transfer ordering and the available bandwidth must agree with the plan. If the required space is unavailable at the scheduled start, the restoration is delayed; the intended completion time then needs to be reassessed. Checking these execution conditions each period is distinct from recomputing all phases and budgets.

Placeholder. Transfer ordering, queueing allowance, and handling of a restoration whose planned allocation or completion time cannot be met.

Returning to computation. An allocated destination is not usable KV until its copy has completed. The next micro-turn must wait for all required ranges to become valid before accessing them. If preparation finishes late, demand restoration recovers the missing state, with the resulting wait charged to service latency. Correct recovery may therefore preserve the history while still violating the soft deadline.

3.4 Revising the Plan

The design relies on small, sufficiently predictable KV increments over the planning horizon. Under this workload assumption, the manager can follow a plan for multiple periods instead of adjusting every eviction budget after every micro-turn. Longer-term growth still changes residency and restoration costs, so the plan needs infrequent correction. Session admission and departure can require larger revisions because they change the set of demands sharing memory and bandwidth.

A revision must account for allocations and copies already in progress. Replacing a future schedule cannot reclaim a referenced block or make an unfinished copy available to computation. Transitions between plans can therefore affect service even when each plan is feasible on its own.

Placeholder. Horizon extension, revision triggers, handling of in-flight work, and service policy when no feasible revised plan exists.

3.5 Correctness Conditions

Residency management preserves the history selected by the application. Every evicted range must remain recoverable, each restored range must contain the state for the same session and logical positions, and no block may be reused while computation or a transfer still references it. Concurrent demand, prefetching, and eviction must maintain one consistent view of allocation and content validity.

These invariants also apply when a transfer misses its deadline or a plan becomes infeasible.

Placeholder. State-transition argument and synchronization rules for concurrent restoration, demand, eviction, and plan revision.

4 Implementation

Placeholder. Verified implementation details: serving engine and version; integration of session phases with execution; KV allocation and host-copy tracking; H2D scheduling and completion events; plan revision and recovery paths. Identify which parts of Section 3 are implemented and which remain design proposals.

Placeholder. Comparison infrastructure: shared execution components, baseline adaptations, timing instrumentation, and measured instrumentation overhead.

5 Evaluation

The evaluation asks whether coordinating phases and KV restoration increases session capacity under a fixed service objective, and which resource limits that increase. It separates the main capacity comparison from the measurements needed to explain it: memory pressure, restoration delay, mechanism ablations, and behavior as histories grow.

Placeholder. Experimental results are pending. Specify the evaluated models, workloads, platforms, software versions, and repetition protocol before reporting measurements.

5.1 Methodology

Workload and comparison controls. A residency comparison holds the model, precision, input content, retained history, and generation work fixed. The controls also match batching limits, scheduling policies, and resource budgets except for the variable under study. Changing the retention policy changes what later computation can access, so comparisons across retention policies require separate quality measurements. Workload parameters include the period, session duration, context-length range, and input and output configuration.

Compared systems. Full residency establishes capacity and latency without recovery. On-demand host restoration exposes the cost of putting recovery on the execution path. Recomputation provides a second way to recover missing state when the required historical inputs are available. Comparisons with PILARIUS use the same generation workload and include the phase and restoration ablations described below.

Placeholder. Named external baselines, exact configurations, host-copy policies, and any adaptations needed for the periodic workload.

Service capacity. The main metric is the largest concurrent session population that satisfies the specified deadline-miss and failure criteria over the observation interval. Offered, admitted, and completed work are reported separately, so refusing more load cannot appear as a latency improvement. Deadline violations use $c(i, k) > r(i, k + 1)$, and latency distributions use $c(i, k) - r(i, k)$. The analysis also records initial phase-alignment delay and accumulated backlog. Progress measurements distinguish legitimate silence from missing generation work, which a timely empty response can conceal.

Placeholder. Allowed miss rate, session-failure criteria, observation interval, load sweep, repetitions, and uncertainty reporting.

Resource measurements. GPU memory measurements include all KV allocations, particularly destinations whose copies have not completed. Valid content, evicted bytes, and host-copy storage are recorded separately. Transfer measurements distinguish waiting for space, queueing for the copy stream, copying, and completion handling. Compute measurements cover the work required by each micro-turn and its interference with transfers. These measurements connect a change in session capacity to the resource it consumes.

5.2 Capacity and Service Latency

The main comparison varies concurrency and session duration to locate the service limit of each system. Latency and deadline violations at equal offered load show whether recovery introduces a cost before capacity is exhausted. At each system's capacity limit, the allocation and execution traces identify whether memory, transfers, computation, or growing backlog prevents further admission.

Placeholder. Supported concurrency and service-latency distributions for all systems, with completed work and failure outcomes.

The full-residency limit. The motivating measurement varies retained history and concurrency together. A memory-limited operating point requires evidence that KV allocation prevents additional service while the corresponding computation still fits within the period. Session-level idle intervals alone do not establish spare compute capacity across the workload. Near the memory boundary, the measurements report the complete workload's execution time per period and the remaining fraction of the period, rather than inferring slack from a device-utilization counter. Calibration and validation use separate configurations to test Equation 5, including regions where it predicts no slack. An additional-work experiment tests whether the observed budget can accommodate more session work or supported recomputation while preserving the service objective.

Placeholder. Joint allocation and execution traces at the full-residency limit, predicted versus measured slack, and additional work completed within the same deadline.

5.3 Mechanism Ablations

The ablations isolate phase assignment from advance restoration. The first comparison holds the restoration policy fixed and changes aligned releases to uniformly staggered releases. The second keeps phases fixed and compares three recovery policies: on-demand restoration, prefetching after submission, and restoration planned before the next tick. Their combination tests whether spreading demand improves the memory and transfer schedule beyond either action alone.

A further comparison holds the phase schedule fixed while varying eviction budgets and restoration lead time. It tests the coupling in Section 2.3: too little eviction leaves memory

occupied, whereas excessive eviction or early allocation can make the schedule infeasible. Host-copy policy, retained history, and transfer implementation remain fixed within each comparison.

Placeholder. Ablation results: session capacity, peak allocation, transfer utilization, deadline violations, and the selected per-session budgets.

Restoration on the critical path. For each micro-turn, the analysis measures how much restoration finishes before its tick and how much delays computation. Allocation waiting, copy-stream queueing, transfer time, and post-transfer scheduling are distinguished. This decomposition tests whether the design removes recovery delay or moves it to an earlier queue.

Placeholder. Restoration timelines and latency decomposition at matched load, including waiting before submission.

5.4 History Growth and Plan Revision

Admission predictions are compared with execution over configurations and traces not used for calibration. The comparison covers per-period KV growth, transfer duration, computation time, and peak allocation. Prediction error is reported against the horizon and scheduling margins used to make the admission decision.

Continuous runs span multiple planning horizons to test service beyond the initial admission decision. One experiment keeps the session population fixed while histories grow; another introduces departures and new admissions. Together they distinguish gradual correction from population changes. The relevant outcomes include revision frequency and cost, service around each revision, and the treatment of sessions when a new feasible plan cannot be found.

Placeholder. Prediction errors, revision behavior, and continuous service across horizons; a successful initial horizon alone does not establish sustained capacity.

5.5 Correctness and Operating Limits

Correctness comparisons preserve the same history and model inputs as the reference execution. They exercise state round trips and overlapping demand, restoration, eviction, and plan changes. For copying, the correctness criterion is preservation of KV content; any recomputation path additionally requires a numerical and output-equivalence criterion.

Placeholder. Correctness criteria and results, including delayed copies, allocation pressure, and concurrent operations.

Overheads. The sensitivity analysis varies the period, retained history, usable KV capacity, effective transfer bandwidth, and host capacity. These experiments identify where recovery consumes the available slack or releases too little memory to increase service capacity. The comparison includes scheduling and host-backup overhead, along with the effect of staggering on batching efficiency.

Placeholder. Measured overheads and limits, including late input and transfer contention.

6 Discussion and Limitations

Applications without phase control. An application may expose future ticks while requiring fixed release phases. Advance restoration can still use that timing information, but the manager cannot redistribute demand by assigning phases. The benefit then depends on the supplied schedule and available memory. The phase ablation in Section 5.3 separates these two sources of benefit.

Finite-horizon admission. A finite prediction horizon limits how far admission can anticipate history growth and interference. Smooth growth can permit infrequent revisions, but it does not guarantee that a feasible schedule exists indefinitely. If state keeps growing, memory, host capacity, or recovery time can eventually become limiting. The reported capacity is consequently tied to session duration, the prediction horizon, and the policy for handling infeasible revisions.

Using spare computation for recovery. Recomputation could extend cyclic restoration when H2D bandwidth becomes limiting. After using a selected KV range, the manager could release it and rebuild it before the next tick from retained historical inputs. This would be a recurring recovery choice, not only a fallback after a failed transfer. Its compute budget must come from slack across the admitted workload; the idle interval of one session is insufficient if other sessions occupy the GPU.

Combining loading with recomputation is established in prior work. Cake recomputes earlier prefix chunks while loading later chunks, exploiting the lower attention cost of earlier positions [3]. For multimodal sessions, applying such a division also requires the appropriate replay inputs, positions, dependencies, and temporary memory. A shorter period reduces the time available for both transfers and recomputation, so it does not by itself favor recomputation.

Placeholder. Cyclic recomputation design: retained replay inputs, eligible ranges, activation conditions, and joint compute, transfer, and memory budgeting.

Workload and platform dependence. A small retained history leaves little memory to reclaim, and a short interval between uses leaves little time to restore it. Effective transfer

bandwidth, host-copy availability, and batching efficiency determine the useful eviction budget. Different periods or execution profiles can also produce unequal demands despite uniformly spaced phases. Uniform spacing spreads release times but does not optimize for these differences in resource demand.

7 Related Work

Scheduling and KV allocation. Orca schedules generation at iteration granularity, allowing batch membership to change as requests arrive and finish [9]. PagedAttention maps logical KV blocks to noncontiguous physical storage, reducing fragmentation and enabling sharing in vLLM [4]. vLLM also supports swapping and recomputation when memory pressure requires preemption. These techniques improve how active work uses computation and memory. The residency problem considered here remains when the valid KV retained by periodic sessions exceeds GPU capacity. PILARIUS coordinates when that state occupies the GPU between uses; it still needs an execution engine and allocator that use the resulting capacity efficiently.

State recovery during interruptions. InferCept manages requests whose generation pauses for interactions with external tools or environments [1]. It estimates interruption durations, chooses among preserving, swapping, and recomputing KV, and chunks recovery work to reduce interference with other requests. This work establishes that idle-state decisions should consider both memory occupancy and recovery cost. Our setting adds a recurring restoration target and, where permitted, control over session phases. The design uses these properties to coordinate repeated recovery with the allocation of its destinations.

Combining loading and recomputation. Cake prepares cached prefixes by concurrently loading some KV chunks and recomputing others [3]. Its division follows causal attention costs: recomputation proceeds from earlier positions while loading proceeds from later positions. The extension in Section 6 builds on this use of computation to supplement loading. Periodic restoration additionally requires fitting repeated preparation into a shared schedule as retained histories grow.

Latency-constrained serving. DistServe separates prefill and decoding onto different GPUs and allocates resources to satisfy time-to-first-token and time-per-output-token objectives [10]. Its design addresses interference between execution stages with different latency requirements. PILARIUS instead coordinates retained state across successive periods of an ongoing session, including time when that session performs no computation. Both designs require evaluating useful service under a latency objective rather than reporting throughput alone.

Periodic interaction and bounded history. Metronome studies memory exhaustion in periodic interaction serving and combines an in-engine KV window with deadline-aware admission [6]. Its window bounds state by retaining recent tokens and attention sinks. Its evaluation also uses phase-staggered input streams. PILARIUS preserves the history selected by the application and varies its GPU residency through partial eviction and advance restoration. This trades host capacity and recurring recovery work for GPU capacity. The approaches therefore make different retention and resource choices; a performance comparison must either hold retained history fixed or report the effect of changing it.

8 Conclusion

Periodic interaction exposes repeated opportunities to release a session's idle KV and restore it before the next tick. Using those opportunities requires coordinating transfer time with the GPU space occupied by restoration destinations. PILARIUS combines uniform phase assignment with per-session eviction and restoration planning over a finite horizon, followed by infrequent revisions as histories and session populations change. The design seeks to increase service capacity while preserving retained history and meeting recurring soft deadlines.

Placeholder. Measured capacity and latency gains, together with the workload, duration, and resource conditions under which they hold.

References

- [1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. 2024. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 81–95. <https://proceedings.mlr.press/v235/abhyankar24a.html>
- [2] Junbo Cui, Bokai Xu, Chongyi Wang, Tianyu Yu, Weiye Sun, Yingjing Xu, Tianran Wang, Zhihui He, Wenshuo Ma, Tianchi Cai, Jiancheng Gui, Luoyuan Zhang, Xian Sun, Fuwei Huang, Moye Chen, Zhuo Lin, Hanyu Liu, Qingxin Gui, Qingzhe Han, Yuyang Wen, Huiping Liu, Rongkang Wang, Yaqi Zhang, Hongliang Wei, Chi Chen, You Li, Kechen Fang, Jie Zhou, Yuxuan Li, Guoyang Zeng, Chaojun Xiao, Yankai Lin, Xu Han, Maosong Sun, Zhiyuan Liu, and Yuan Yao. 2026. MiniCPM-o 4.5: Towards Real-Time Full-Duplex Omni-Modal Interaction. arXiv:2604.27393 [cs.CL] doi:10.48550/arXiv.2604.27393
- [3] Shuowei Jin, Xueshen Liu, Qingzhao Zhang, and Z. Morley Mao. 2025. Compute Or Load KV Cache? Why Not Both? arXiv:2410.03065 [cs.LG] doi:10.48550/arXiv.2410.03065
- [4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. Association for Computing Machinery, 611–626. doi:10.1145/3600006.3613165
- [5] C. L. Liu and James W. Layland. 1973. Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. *J. ACM* 20, 1 (1973), 46–61. <https://www.cis.upenn.edu/~lee/07cis505/Lec/liu73scheduling.pdf>

- [6] Jiaying Meng and Bojie Li. 2026. Metronome: Bound the Cache, Keep the Beat for Real-Time Interaction Model Serving. arXiv:2607.02640 [cs.SD] doi:10.48550/arXiv.2607.02640
- [7] OpenBMB. 2026. MiniCPM-o-4_5 Model Configuration. https://huggingface.co/openbmb/MiniCPM-o-4_5/blob/503e754207c94da6bb26850b4469f367c9ea3582/config.json
- [8] Thinking Machines Lab. 2026. Interaction Models: A Scalable Approach to Human-AI Collaboration. *Thinking Machines Lab: Connectionism* (May 2026). doi:10.64434/tml.20260511
- [9] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, 521–538. <https://www.usenix.org/conference/osdi22/presentation/you>
- [10] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>